

NEPAL COLLEGE OF INFORMATION TECHNOLOGY

Balkumari, Lalitpur

Affiliated to Pokhara University

LAB REPORT FOR EMBEDDED SYSTEMS



LAB REPORT 9: VHDL CODE FOR FLIP FLOP

Submitted by:

Name: Arpan Adhikari

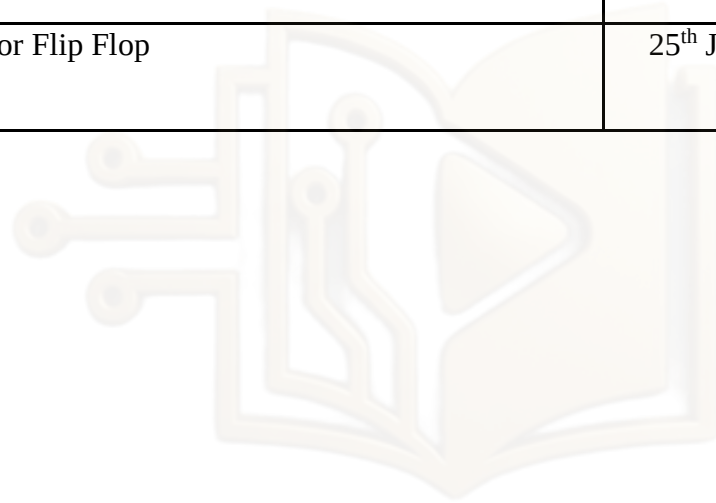
Roll No.: 231309

BE-CE (5th SEM)

Submitted to:

Er. Simanta Kasaju

S. N	LAB TOPIC	DATE	REMARKS
1.	VHDL Code (OR & AND Gate)	20 th Nov, 2025	
2.	VHDL Code (OR & NOT Gate) using testbench	23 rd Nov	
3.	Universal Gates and Special Gates	30 th Nov	
4.	VHDL Code for Adder	7 th Dec	
5.	VHDL Code for Subtractor	14 th Dec	
6.	VHDL Code for Multiplexer	21 st Dec	
7.	VHDL Code for De-Multiplexer	4 th Jan, 2026	
8.	VHDL Code for Decoder and Encoder	18 th Jan	
9.	VHDL Code for Flip Flop	25 th Jan	



LAB 9: VHDL CODE FOR FLIP FLOP

Theory

A **flip-flop** is a **basic sequential logic circuit** capable of storing **one bit of binary information**. Unlike combinational circuits, flip-flops have **memory**, meaning their output depends on the **present input and the previous state**. Flip-flops are triggered by a **clock signal** and are widely used in **registers, counters, and memory elements**.

SR (Set-Reset) Flip-Flop

Theory

The **SR flip-flop** has two inputs: **S (Set)** and **R (Reset)**, along with outputs **Q** and $\bar{Q}$.

- When **S = 1**, the output is set to 1
- When **R = 1**, the output is reset to 0
- When both inputs are 0, the output remains unchanged
- When both inputs are 1, the condition is **invalid**

Truth Table (SR Flip-Flop)

S	R	Q (Next State)
0	0	No change
0	1	0
1	0	1
1	1	Invalid

Limitation

The **invalid condition (S = R = 1)** is a major drawback of the SR flip-flop.

JK Flip-Flop

Theory

The **JK flip-flop** is an improved version of the SR flip-flop that **eliminates the invalid state**. It has two inputs: **J** and **K**.

- When both inputs are 0, the output remains unchanged
- When **J = 1** and **K = 0**, the output is set
- When **J = 0** and **K = 1**, the output is reset
- When **J = K = 1**, the output **toggles**

Truth Table (JK Flip-Flop)

J	K	Q (Next State)
0	0	No change
0	1	0
1	0	1
1	1	Toggle

Advantage

The JK flip-flop has **no invalid condition** and is widely used in **counters and registers**.

D (Data or Delay) Flip-Flop

Theory

The **D flip-flop** has a single input **D** and eliminates ambiguity by ensuring that the output follows the input at the clock edge.

- The output **Q** takes the value of **D** when the clock is triggered
- It is commonly used for **data storage and registers**

Truth Table (D Flip-Flop)

D	Q (Next State)
0	0
1	1

Advantage

Simple operation and **no invalid state**.

T (Toggle) Flip-Flop

Theory

The **T flip-flop** is obtained from a JK flip-flop by tying **J and K together**.

- When **T = 0**, the output remains unchanged
 - When **T = 1**, the output toggles
- It is mainly used in **binary counters**.

Truth Table (T Flip-Flop)

T	Q (Next State)
0	No change
1	Toggle

Master-Slave Flip-Flop

Theory

A **Master-Slave flip-flop** consists of two flip-flops connected in series to eliminate **race-around condition**.

- The **master** responds during the first half of the clock
 - The **slave** responds during the second half
- This ensures stable output changes.
-

Comparison of Flip-Flops

Flip-Flop	Inputs	Special Feature
SR	S, R	Simple but has invalid state
JK	J, K	No invalid state
D	D	Output follows input
T	T	Toggles output
Master-Slave	Varies	Eliminates race condition

Applications of Flip-Flops

- Registers
- Counters
- Memory units
- Shift registers
- Timing and control circuits



SR FLIP FLOP

BEHAVIORAL:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity arp_flipflop is
port(
    S,R, Clk : in Std_logic;
    Q, Qn : out Std_logic
);
end arp_flipflop;

architecture arch_arp of arp_flipflop is
signal preQ : Std_logic := '0';

begin
    process(Clk, S, R)
    begin
        if (Clk = '1') then
            if(S='0' and R='0') then
                preQ <= preQ;
            elsif (S='0' and R='1') then
                preQ <= '0';
            elsif (S='1' and R='0') then
                preQ <= '1';
            else
                preQ <= 'X';
            end if;
        end if;
    end process;

    Q <= preQ;
    Qn <= not preQ;

end arch_arp;
```

```
library IEEE;
use IEEE.std_logic_1164.all;

entity arp_flipflop is
port(
    S,R, Clk : in Std_logic;
    Q, Qn : out Std_logic
);
end arp_flipflop;

architecture arch_arp of arp_flipflop is
signal preQ : Std_logic := '0';

begin
    process(Clk, S, R)
    begin
        if (Clk = '1') then
            if(S='0' and R='0') then
                preQ <= preQ;
            elsif (S='0' and R='1') then
                preQ <= '0';
            elsif (S='1' and R='0') then
                preQ <= '1';
            else
                preQ <= 'X';
            end if;
        end if;
    end process;

    Q <= preQ;
    Qn <= not preQ;
end arch_arp;
```



TESTBENCH:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity test_flipflop is
end test_flipflop;

architecture test_flipflop_arch of test_flipflop is
component arp_flipflop is
port(
    S,R, Clk : in Std_logic;
    Q, Qn : out Std_logic
);
end component;

signal S, R, Clk : Std_logic;
signal Q, Qn : Std_logic;

begin
    UUT: arp_flipflop port map
    (
        S => S,
        R => R,
        Clk => Clk,
        Q => Q,
        Qn => Qn
    );

    clk_process : process
    begin
        Clk <= '0';
        wait for 0 ps;
        Clk <= '1';
        wait for 5 ps;
    end process;

    stim_proc : process
    begin
        S <= '0'; R <= '0';
        wait for 100 ps;
        S <= '0'; R <= '1';
        wait for 100 ps;
        S <= '1'; R <= '0';
        wait for 100 ps;
        S <= '1'; R <= '1';
        wait for 100 ps;
    end process;

end test_flipflop_arch;
```




```

library IEEE;
use IEEE.std_logic_1164.all;

entity test_flipflop is
end test_flipflop;

architecture test_flipflop_arch of test_flipflop is
component arp_flipflop is
port(
    S,R, Clk : in Std_logic;
    Q, Qn : out Std_logic
);
end component;

signal S, R, Clk : Std_logic;
signal Q, Qn : Std_logic;

begin
    UUT: arp_flipflop port map
    (
        S => S,
        R => R,
        Clk => Clk,
        Q => Q,
        Qn => Qn
    );

    clk_process : process
    begin
        Clk <= '0';
        wait for 0 ps;
        Clk <= '1';
        wait for 5 ps;
    end process;

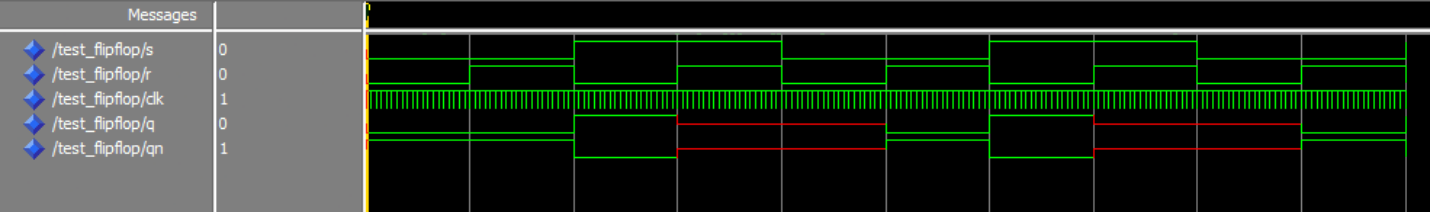
    stim_proc : process
    begin
        S <= '0'; R <= '0';
        wait for 100 ps;
        S <= '0'; R <= '1';
        wait for 100 ps;
        S <= '1'; R <= '0';
        wait for 100 ps;
        S <= '1'; R <= '1';
        wait for 100 ps;
    end process;

end test_flipflop_arch;

```



WAVEFORM:



JK FLIP FLOP

BEHAVIORAL:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity arp_flipflop is
port(
    S,R, Clk : in Std_logic;
    Q, Qn : out Std_logic
);
end arp_flipflop;

architecture arch_arp of arp_flipflop is
signal preQ : Std_logic := '0';

begin
    process(Clk, S, R)
    begin
        if (Clk = '1') then
            if(S='0' and R='0') then
                preQ <= preQ;
            elsif (S='0' and R='1') then
                preQ <= '0';
            elsif (S='1' and R='0') then
                preQ <= '1';
            else
                preQ <= not preQ;
            end if;
        end if;
    end process;

    Q <= preQ;
    Qn <= not preQ;

end arch_arp;
```

TESTBENCH:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity test_flipflop is
end test_flipflop;

architecture test_flipflop_arch of test_flipflop is
component arp_flipflop is
port(
    S,R, Clk : in Std_logic;
    Q, Qn : out Std_logic
);
end component;

signal S, R, Clk : Std_logic;
signal Q, Qn : Std_logic;

begin
    UUT: arp_flipflop port map
    (
        S => S,
        R => R,
        Clk => Clk,
        Q => Q,
        Qn => Qn
    );

    clk_process : process
    begin
        Clk <= '0';
        wait for 0 ps;
        Clk <= '1';
        wait for 5 ps;
    end process;

    stim_proc : process
    begin
        S <= '0'; R <= '0';
        wait for 100 ps;
        S <= '0'; R <= '1';
        wait for 100 ps;
        S <= '1'; R <= '0';
        wait for 100 ps;
        S <= '1'; R <= '1';
        wait for 100 ps;
    end process;

end test_flipflop_arch;
```



D FLIP-FLOP

BEHAVIORAL

```
library IEEE;
use IEEE.std_logic_1164.all;

entity arp_dflipflop is
port(
    D, Clk : in std_logic;
    Q, Qn : out std_logic
);
end arp_dflipflop;

architecture arch_arp of arp_dflipflop is
    signal preQ : std_logic := '0';
begin
    process(Clk, D)
    begin
        if (Clk = '1') then
            preQ <= D;
        end if;
    end process;

    Q <= preQ;
    Qn <= not preQ;
end arch_arp;
```

TESTBENCH

```
library IEEE;
use IEEE.std_logic_1164.all;

entity test_dflipflop is
end test_dflipflop;

architecture test_arch of test_dflipflop is
component arp_dflipflop
port(
    D, Clk : in std_logic;
    Q, Qn : out std_logic
);
end component;

signal D, Clk : std_logic;
signal Q, Qn : std_logic;

begin
UUT: arp_dflipflop port map (D => D, Clk => Clk, Q => Q, Qn => Qn);

clk_process : process
begin
    Clk <= '0'; wait for 5 ps;
    Clk <= '1'; wait for 5 ps;
end process;

stim_proc : process
begin
    D <= '0'; wait for 100 ps;
    D <= '1'; wait for 100 ps;
    D <= '0'; wait for 100 ps;
end process;
end test_arch;
```

T FLIP-FLOP

BEHAVIORAL

```
library IEEE;
use IEEE.std_logic_1164.all;

entity arp_tflipflop is
port(
    T, Clk : in std_logic;
    Q, Qn : out std_logic
);
end arp_tflipflop;

architecture arch_arp of arp_tflipflop is
    signal preQ : std_logic := '0';
begin
    process(Clk, T)
    begin
        if (Clk = '1') then
            if (T = '1') then
                preQ <= not preQ;
            else
                preQ <= preQ;
            end if;
        end if;
    end process;

    Q <= preQ;
    Qn <= not preQ;
end arch_arp;
```

TESTBENCH

```
library IEEE;
use IEEE.std_logic_1164.all;

entity test_tflipflop is
end test_tflipflop;

architecture test_arch of test_tflipflop is
component arp_tflipflop
port(
    T, Clk : in std_logic;
    Q, Qn : out std_logic
);
end component;

signal T, Clk : std_logic;
signal Q, Qn : std_logic;

begin
UUT: arp_tflipflop port map (T => T, Clk => Clk, Q => Q, Qn => Qn);

clk_process : process
begin
    Clk <= '0'; wait for 5 ps;
    Clk <= '1'; wait for 5 ps;
end process;

stim_proc : process
begin
    T <= '0'; wait for 100 ps;
    T <= '1'; wait for 100 ps;
    T <= '1'; wait for 100 ps;
    T <= '0'; wait for 100 ps;
end process;
end test_arch;
```


Result

The **SR, JK, D, and T flip-flops** were successfully designed and simulated using **VHDL behavioral modeling**. The output waveforms obtained from the simulations matched the expected **truth tables**. Each flip-flop responded correctly to the clock signal, and the stored output changed according to the applied input conditions.

Conclusion

This experiment successfully demonstrated the design and operation of **basic sequential circuits** using **VHDL**. The behavior of different flip-flops and their role in **data storage and state control** were clearly understood. The experiment confirmed that VHDL is an effective tool for modeling, simulating, and verifying **flip-flop based digital circuits**.

